Ders 2 Çalışma Özeti

Ders 2 Çalışma Özeti

Genel Konular

- Dersin içeriği ve grup projesi kısaca tekrar edilir. Projenin teslim takvimi verilir.
- Yazılım mühendisliğinin tanımı, katmanlı yapısı ve geleneksel süreç modelleri (RAD, spiral) hatırlatılır.
- Çevik yaklaşımlar, XP (Extreme Programming) ve test güdümlü geliştirme (Test Driven Development, TDD) kalite açısından ele alınır.
- Kaliteli yazılımın tanımı ve McCall kalite modeli (kalite üçgeni, kalite faktörleri ve ölçütler) işlenir.
- Kalite sağlama aktiviteleri sayılır. Doğrulama (verification) ve geçerleme (validation) ayrımı yapılır.
- Kalite maliyeti: hatayı önleme maliyeti, iç ve dış hata maliyeti, hatanın bulunduğu aşamaya göre düzeltme maliyeti.
- Yazılım standartları ve kalite yönetim sistemi anlatılır.
- Gözden geçirme ve onaylama (review): amacı, ne olmadığı, toplantı rolleri, gözden geçirme tipleri ve seçenek matrisi, hangi aşamada neyin gözden geçirildiği.

Hocanın Özellikle Vurguladığı Kısımlar

- Hatayı ne kadar erken ararsanız bulma olasılığı o kadar artar ve düzeltme maliyeti o kadar düşer.
- Kaliteyi baştan sağlamanın (hatayı önlemenin) maliyeti, hatayı sonradan bulup düzeltmenin maliyetinden her zaman azdır. Kaliteyi test aşamasına bırakmayın.
- Doğrulama ve geçerleme Türkçede benzer kelimelerdir ama anlamları farklıdır:
  - Doğrulama (verification): yazılımı doğru yolla ürettik mi?
  - Geçerleme (validation): istenen ürünü ürettik mi?
- Yazılım mühendisliği “en ucuz” yazılımı değil, en iyi yazılımı en uygun maliyetle elde etmeyi hedefler. Bunun için proje yönetimi de kalite yönetimi de gerekir.
- Gözden geçirmeyi yazılımı geliştiren kişi değil, üçüncü bir göz yapmalıdır. Geliştirici kendi hatasını tekrarlama eğilimindedir.
- Gözden geçirme aynı zamanda bir eğitim ortamıdır. Kodu veya tasarımı hazırlayan kişi hatasını zamanında öğrenir.
- Çevik yaklaşımlar yaygınlaştı ama geleneksel modeller kaybolmadı. Hata kritik yazılımlarda spiral model, var olan bir yazılımın yenilenmesinde şelale (waterfall) modeli hâlâ kullanılır.

Kısa Tekrar Notları

- Kaliteli yazılım: önceden belirlenmiş işlevsel gereksinimlere ve geliştirme standartlarına uygun, kabul edilebilir hata düzeyinde olan yazılımdır.
- McCall kalite üçgeninin üç kenarı:
  - Ürün kullanımı (product operation): doğruluk (correctness), güvenirlik (reliability), kullanışlılık (usability), bütünlük (integrity), etkinlik (efficiency).
  - Ürün geçişi (product transition): taşınabilirlik (portability), yeniden kullanılabilirlik (reusability), birlikte çalışabilirlik (interoperability).
  - Üründe değişiklik (product revision): bakım yapılabilirlik (maintainability), esneklik (flexibility), test edilebilirlik (testability).

- Kalite faktörleri doğrudan ölçülemez. Ölçülebilir ölçütlere bağlı doğrusal bir regresyon denklemiyle kestirilir: $F_q = c_1 m_1 + c_2 m_2 + \cdots + c_n m_n$.
- Hata düzeltme göreli maliyeti: gereksinim analizi 0,2; tasarım 0,5; birim testi 2; kabul testi 5; bakım 20.
- Hataların yaklaşık %65'i tasarım aşamasında yapılır. Gözden geçirmede bunların yaklaşık %75'i bulunur.
- Hata maliyeti:
  - İç hata maliyeti: hata yazılım kullanıma alınmadan, ekip içinde bulunur.
  - Dış hata maliyeti: hata kullanımdan sonra kullanıcıya yansır. Müşterinin güveni de kaybedilir, bu yüzden daha pahalıdır.
- Gözden geçirme tipleri: formal olmayan gözden geçirme (informal review), yapısal denetim (walkthrough), inceleme (inspection), round robin peer review.
- Statik kalite değerlendirme: yazılım çalıştırılmadan yapılır (ör. inceleme). Dinamik kalite değerlendirme: yazılım çalıştırılarak yapılır (test).

Detaylı Açıklamalar

Geleneksel modellerin hatırlatılması

- Geleneksel modeller: şelale (waterfall), artımlı (incremental), spiral, RAD, birleşik süreç (Unified Process). Unified Process nesneye yönelik yaklaşımı en iyi destekleyen modeldir.
- RAD (Rapid Application Development): yazılım modüllere bölünür, her modülü ayrı bir grup geliştirir, sonra modüller birleştirilip test edilir. Kullanım şartları: geniş ekip olanağı olmalı ve yazılım modüllere ayrılabilmelidir.
- Spiral (sarmal) model evrimsel bir modeldir. Her çevrim şu aşamalardan geçer: müşteriyle iletişim, planlama, risk analizi, mühendislik, yapım (construction) ve teslim, müşteri değerlendirmesi. İlk çevrimde prototip, sonraki çevrimlerde yeni ürün ve ürüne yapılan eklemeler geliştirilir.

Çevik yaklaşımlar ve kalite

- Çevik yaklaşımlar artan müşteri taleplerini daha hızlı karşılamak için geleneksel modellere alternatif olarak ortaya çıktı. Amaç gereksinimlerin geç ortaya çıkmasını önlemektir.
- XP dört değer üzerine kuruludur: basitlik, iletişim, geri besleme, cesaret.
- XP'nin öne çıkan uygulamaları: eşli programlama (pair programming), sürekli yeniden tasarım (refactoring), ortak kod mülkiyeti, sürekli bütünleştirme (continuous integration).
- Çevik modellerde kalite açısından farklar:
  - Dinamik kalite değerlendirme (test) statik değerlendirmeden daha çok kullanılır.
  - Kalite aktiviteleri sürecin erken adımlarında başlar ve daha sık yapılır.
  - Kullanıcı hikayeleri (user stories), birim testleri ve müşteri geri beslemesi sürekli kalite değerlendirmesi sağlar.

Test güdümlü geliştirme (TDD)

- Küçük ve hızlı iterasyonlarla, önceden hazırlanmış testler yazılarak ilerleyen bir geliştirme yöntemidir.
- Sistem analizi sırasında yazılımın nasıl test edileceği ve kullanıcı testinin planı da hazırlanır.
- Adımlar:
  - Yazılımın en küçük parçası (birim) için test senaryolarını (test case) yaz.
  - Kod henüz yazılmadığı için testin başarısız olduğunu gör.
  - Kodu yaz ve birimleri çalıştırarak test et. Bütünlük testini bekleme.
  - Test case'lerin geçtiğini gör.
  - Hata çıktıkça kodu yeniden düzenle (refactor) ve tekrar kullan.
- Hoca, TDD üzerine erken çalışmalar yapan Hakan Erdoğmuş'un yaklaşık 10 yıl önce bölümün düzenlediği Ulusal Yazılım Mühendisliği Sempozyumu'na davetli konuşmacı olarak geldiğini anlatır. Bu çalışmalara göre TDD programcı üretkenliğini artırır, ama kalitede bazı eksikler görülmüştür. Son 10 yılda TDD üzerine iyileştirme çalışmaları sürmektedir. TDD sektörde yaygın kullanılır.

McCall kalite özellikleri

Özellik	Açıklama
Doğruluk (correctness)	Spesifikasyona uygunluk, müşteri isteklerini karşılama derecesi
Güvenirlik (reliability)	Tasarlanan işlemleri ne kadar duyarlılıkla yerine getirdiği; doğru sonuca ne kadar güvenilebildiği
Verimlilik (efficiency)	İşlevler için gereken bilgi işlem kaynağı ve kodlama gideri ile işlevlerin uyumu
Güvenlik / bütünlük (security, integrity)	Yetkisiz kişilerin yazılıma girmesinin engellenmesi
Kullanışlılık (usability)	Öğrenme, işletme, girdi hazırlama ve çıktı yorumlama kolaylığı; kullanıcı dostu olma
Bakım yapılabilirlik (maintainability)	Hatanın yerini bulup düzeltme kolaylığı
Esneklik (flexibility)	Yazılımda değişiklik yapma kolaylığı
Test edilebilirlik (testability)	Yazılımın sınanabilme kolaylığı
Taşınabilirlik (portability)	Farklı donanım veya işletim sisteminde kullanılabilme
Yeniden kullanılabilirlik (reusability)	Yazılımın tümünün, bir parçasının veya tasarımının başka uygulamada kullanılabilmesi
Birlikte çalışabilirlik (interoperability)	Başka bir sistemle birlikte çalışabilme, veri alışverişi yapabilme

- Hoca, interoperability için daha önce “bağlanabilirlik” dendiğini, ama “birlikte çalışabilirlik” teriminin anlamı daha iyi verdiğini söyler.
- Yeniden kullanılabilirlik yazılım mühendisliğinin önemli bir alt alanıdır. Nesneye yönelik yaklaşım, tasarım kalıpları ve alana özgü modelleme (domain-specific modeling) yeniden kullanımı destekler.

Kalite faktörleri ile ölçütler arasındaki ilişki (McCall modeli)

Slayttaki şekilde solda kalite faktörleri, sağda daha ölçülebilir ölçütler vardır. Bir faktör birden çok ölçüte bağlıdır ve her ölçütün katkı oranı farklıdır. Hocanın derste verdiği eşleştirmeler:

- Doğruluk: izlenebilirlik (traceability), tam olma (completeness), uyumluluk (consistency).
- Güvenirlik: doğruluk (accuracy), uyumluluk, hata toleransı (error tolerance). Hata toleransı, hata çıktığında yazılımın çalışmaya devam edebilmesidir.
- Etkinlik: çalışma etkinliği (hız, yanıt süresi), bellek etkinliği.
- Bütünlük: erişim kontrolü, erişim denetimi.
- Kullanılabilirlik: çalışabilirlik (operability), eğitim, iletişim yapılabilirlik.
- Bakım yapılabilirlik: basitlik, büyük olasılıkla özlülük (conciseness; altyazıda “hassasiyet” gibi duyuluyor), ifade edilebilirlik (belgeleme var mı?), modülerlik.
- Test edilebilirlik: basitlik, araç desteği (instrumentation), ifade edilebilirlik, modülerlik.
- Yeniden kullanılabilirlik: basitlik, genelleştirilebilirlik (tasarım kalıpları gibi), yazılım sistemi bağımsızlığı, büyük olasılıkla makine bağımsızlığı (altyazıda net duyulmuyor), modülerlik. Başka modüllere çok bağımlı bir modülün yeniden kullanılabilirliği düşüktür.
- Birlikte çalışabilirlik: modülerlik, iletişim genelliği, veri genelliği (başka veri tabanlarını kullanabilme).

Kalite sağlama aktiviteleri

- Yazılım standartlarını kullanmak.
- Doğrulama ve geçerleme yapmak.
- Gözden geçirme (review) ve inceleme (inspection) ile ara ürünleri denetlemek.
- Formal bir yazılım geliştirme süreç modeli benimsemek.
- Dinamik değerlendirme: test.

- Hata ve kusur analizi, değişim yönetimi, konfigürasyon yönetimi, kullanıcı ve geliştirici eğitimi, risk yönetimi.

Süreç adımlarında kalite için dikkat edilecekler:

- Gereksinim analizi: sistemin kolay öğrenilip kullanılmasına çaba gösterin.
- Genel (mimari) tasarım: modülleri ayırırken taşınabilirlik, birlikte çalışabilirlik ve modül bağımsızlığına önem verin.
- Ayrıntılı tasarım ve gerçekleştirme: doğruluk faktörüne ağırlık verin. Her modülün algoritması doğru çalışmalıdır.
- Test: doğruluk ve performans gibi faktörleri ölçmeye çalışın.

Kaliteyi belirlemek için doğrudan nesnel ölçütler kullanılabilir (McCabe ile karmaşıklık, Halstead ile işlem sayısı). Dolaylı ölçütler ve uzman yazılımcıların görüşü de kullanılabilir.

Kalite maliyeti

- Slayttaki grafik: yazılım geliştirme süreci ilerledikçe hata bulma gideri artar, hata bulma olasılığı düşer. Bu yüzden gözden geçirme, süreç modeli ve standart kullanımı önemlidir.
- Hata önleme maliyetine girenler: kalite planlama, formal teknik gözden geçirme ve inceleme, TDD ile baştan yapılan küçük testler, kullanıcı ve geliştirici eğitimi.
- Bakım aşamasında hatayı düzeltmek en pahalısıdır, çünkü tüm yazılımı yeniden anlamak gerekir.
- Hoca kalite maliyet modeli şeklini yalnızca bilgi için gösterir; konunun ekonomi tarafına girilmez.

Yazılım standartları ve kalite yönetim sistemi

- Derste öğrencilerin verdiği örnekler: ISO, CMMI, IEEE. Hoca PMI'ın (Project Management Institute) yalnızca yazılım değil, tüm projeler için proje yönetimi standartları içerdiğini ekler.
- Standartların sınıflandırılması:
  - Geliştirme aşamasına göre: proje planı, gereksinim tanımı, tasarım spesifikasyonu, kodlama, konfigürasyon yönetimi standartları.
  - Kaynağına göre: uluslararası (ISO, IEEE), ulusal (ör. Türk Standartları Enstitüsü), askeri (DoD, ABD Savunma Bakanlığı).
  - Değerlendirme ölçü tipine göre.
- Kalite yönetim sistemi kurulurken bir veya birkaç standart (ör. ISO 9126, DoD) temel alınır. Bir kalite el kitabı hazırlanır. Kurumun standartları ve kalite kontrolleri burada tanımlanır. Her projede bu temel kullanılır ve projeye özel ek standart gerekip gerekmediğine karar verilir.
- Yazılım kalite yönetimi şunları belirler: hangi standart, hangi süreç modeli, neyin ve nasıl ölçüleceği, konfigürasyon yönetiminin nasıl yapılacağı, doğrulama ve geçerleme için neler yapılacağı.
- Kalite yönetiminin faydası: kaliteyi güvence altına almak, ölçümle kalite derecesini belirlemek ve sonraki projelerin tahminine temel oluşturmak.

Gözden geçirme ve onaylama

- Her basamak tamamlanınca durak noktalarında (milestone) yapılan işler gözden geçirilir ve onaylanır. Şelale modelinde bu her adımın sonundadır. Diğer modellerde daha sık aralıklarla yapılır.
- Gözden geçirme bir eleme işlemidir. Hata ve eksikleri zamanında bulup gidermeyi sağlar.
- Amacı:
  - İşlevsel ve mantıksal hataları, kodlama dilinin hatalı kullanımını bulmak.
  - Yazılımın her adımda gereksinimlere uygunluğunu onaylamak.
  - Belirlenen standartlara uygunluğu kontrol etmek. Örnek: gereksinim belgesi IEEE SRS (Software Requirements Specification) standardına uygun mu?
  - Proje yönetimini kolaylaştırmak ve müşteriye verilecek belgelerin kalitesini artırmak.

- Gözden geçirme nedir? Teknik uzmanların çalışmalarının teknik uzmanlar tarafından değerlendirildiği toplantı veya belgeler üzerinden yapılan inceleme. Sonunda gözden geçirme raporu ve hata raporu hazırlanır.
- Gözden geçirme ne değildir?
  - Geliştiricinin yaptığı işi uzun uzun anlattığı bir proje özeti değildir.
  - Ekibin politik veya kişisel değerlendirmesi değildir.
  - Yalnızca bilgi verme toplantısı değildir.
- Gözden geçirme toplantısındaki roller: gözden geçirme lideri, kullanıcı temsilcisi, kalite güvence ekibinden standartları kontrol eden kişi, kayıtçı, üretici (geliştirici ekipten biri). Küçük ekiplerde bazı roller olmayabilir. Örneğin ayrı kalite güvence ekibi yoksa standartları ekip kendi içinde kontrol eder.

Gözden geçirme seçenek matrisi (GG: formal olmayan gözden geçirme, WT: walkthrough, İnc.: inspection):

	GG	WT	İnc.
Gözden geçirme lideri	yok	var	var
Toplantı takvimi	belki	var	var
Gözden geçirenlerin hazırlığı	belki	var	var
Yazılımcı sunum yapıyor mu	belki	var	yok
Hataları bulmak için checklist	yok	yok	var
Bulunan hatalar sınıflanıyor mu	yok	yok	var
Kayıtçı not alıyor mu	belki	var	var

- Matrisin bazı hücreleri altyazıda net duyulmuyor; bunlar büyük olasılıkla slayttandır.
- Inspection'da yazılımcı sunum yapmaz. İnceleyenler raporlar ve kontrol listesi (checklist) üzerinden çalışır.
- Her projede inspection yapılmaz. Formal olmayan gözden geçirme çok daha sık yapılır.
- Küçük projelerde düzeltmeyi geliştirici de yapabilir. Ama eksikler üçüncü bir kişi tarafından daha kolay görülür.

Hangi aşamada ne gözden geçirilir?

Gözden geçirme genelde şu durak noktalarında yapılır: sistem analizi, yazılım geliştirme planı, gereksinim analizi, tasarım, kodlama, test ve bakım-onarım sonunda. Büyük projelerde daha sık aralıklar seçilir.

- Sistem analizi: işlev ve görevlerin dağılımının doğruluğu, ana işlevler, ara birimler, performans ve tasarım kısıtları, alternatifler arasındaki seçimin tutarlılığı, teknik yapılabilirlik (feasibility). Kullanıcıyla bağlam (context) ve birinci düzey veri akış diyagramları üzerinden "sistemi doğru anladık mı?" diye onaylatılır.
- Yazılım geliştirme planı: sorunun ve terimlerin doğru tanımı, kaynakların yeterliliği, görevlerin uygunluğu, gider ve zamanlama tahminlerinin tutarlılığı, riskler.
- Gereksinim analizi: sistem düzeyinde tutarlılık ve doğruluk, sorunun doğru bölümlenmesi, işlevsel olmayan gereksinimlerin yapılabilirliği, kullanıcının prototip üzerinde onayı, zaman, kaynak ve bütçe açısından yapılabilirlik.
- Tasarım:
  - Ön (genel) tasarım: gereksinimler tasarıma ve yazılım mimarisine doğru aktarılmış mı? Modüller etkin ve işlevsel olarak bağımsız mı? Klasik yaklaşımda yapı diyagramı ve modül hiyerarşisi, nesneye yönelik yaklaşımda tasarım sınıf diyagramları ve etkileşimler (mesajlar) incelenir. Bu inceleme kullanıcı tarafından değil, yazılım uzmanları tarafından yapılır.
  - Ayrıntılı tasarım: her modülün algoritması doğru mu?
- Kodlama: tasarım seçilen programlama diline doğru ve algoritmaya uygun aktarılmış mı?
- Test: test işlemleri test planına göre ve hedefe uygun yapılmış mı?

Proje takvimi ve duyurular

- Grup 4-5 kişidir. Hoca 5 kişiyi önerir; 4'ten az olmamalıdır.

- İlk teslim e-postayla yapılır: grup üyeleri ve seçilen yazılım. Yazılımın kendisi gönderilmez, ama kaynak kodu olmalıdır, çünkü bir birim testi istenecektir. Yazılım bir üyenin ara projesi, başka bir dersin projesi veya internetten bulunan kaynak kodlu bir yazılım olabilir.
- Grup listesi verildikten sonra gruba yeni üye eklenemez.
- Takvim: 3. hafta grubunu kuran gönderebilir; 4. hafta grup ve yazılım için son tarihtir; 6. hafta gereksinim raporu teslim edilir.
- Test raporu (test planı, test senaryoları ve test sonuç raporu) son dersten 3 hafta önce teslim edilir. Sunumlar son iki haftada yapılacaktır.
- Gereksinim raporunun nasıl yazılacağı derste ayrıca özetlenecektir.
- Slaytlar Avesis'e değil, yeni ders sisteminin paylaşım alanına konacaktır.
- Hoca bir sonraki derste inceleme (inspection) ve ardından doğrulama-geçerleme konularına geçeceğini söyler.
- Hoca ayrıca Çok Disiplinli Tasarım Projesi için Cumartesi günü bir tanıtım dersi yapacağını duyurur.

Ders sonrası notlar

- Hoca bir yerde "hata kritik askeri yazılımlarda da çevik yaklaşımlar kullanılıyor" der. Hemen ardından spiral modelin hata kritik yazılımlarda kullanıldığını söyler. Bağlama göre kastedilen, bu tür yazılımlarda geleneksel modellerin hâlâ kullanıldığıdır.
- Hoca çevik yaklaşımların 1999 sonu-2000 başında bir manifesto ile başladığını söyler. XP 1990'ların sonunda ortaya çıkmıştır; Çevik Manifesto (Agile Manifesto) 2001'de yayımlanmıştır.
- ISO 9126 yazılım ürün kalitesi standardıdır. Günümüzde yerini büyük olasılıkla ISO/IEC 25010 almıştır.
- Whisper bazı adları bozmuştur: "MacGol/Makkol/McColl" McCall, "maccape/makyip" McCabe, "hashtag/high stat" Halstead, "mental model" incremental (artımlı) model, "Mindstone" milestone, "Kamba" Kanban.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.